Final Report

Project Title-

Employee Salary Prediction Using Regression Based Machine Learning

---

Problem Id: 03

Student Name-Ansh Gupta (Leader), Agamya Saini, Amritaanshu Dheeman

University Roll number-2028391, 2028385, 2028390

Section-L

- **Problem Description:**

In today's organizations, deciding employee salary is not a simple task because salary depends on many important factors such as experience, education level, job role, department, performance rating, certifications, skills, working hours, leadership ability, and job satisfaction.

In most companies, salary decisions are often made manually by HR departments or managers, which can sometimes lead to inconsistency, bias, and lack of transparency. This makes it difficult to ensure fair and accurate salary distribution among employees.To solve this problem, this project aims to build a machine learning-based employee salary prediction system that can predict salary in a more reliable, consistent, and data-driven manner.

In Phase 1, the project focused on designing the complete machine learning pipeline and understanding the theoretical concepts involved in salary prediction using regression models. A synthetic dataset structure with 20 employee-related features was studied to simulate a real organizational environment. Important concepts such as Exploratory Data Analysis (EDA), data cleaning, outlier detection, encoding, normalization,&feature selection were planned and analyzed theoretically.

In Phase 2, the practical implementation of the same machine learning pipeline was performed on dataset contains 1500 employee records. The dataset includes multiple employee-related features such as age, experience, education level, department, job role, performance rating, certifications, overtime, work hours, leadership score, skill match score, company size, and job satisfaction.

The complete implementation includes data cleaning, handling missing values, removing duplicates, outlier detection using the IQR method, encoding categorical variables using One-Hot Encoding, feature scaling using StandardScaler and MinMaxScaler, and forward feature selection for selecting the most important features affecting salary prediction.

Three regression models were implemented and compared:

->Linear Regression

-> Ridge Regression

->Lasso Regression

The models were evaluated using important regression metrics such as R^2 Score, RMSE & MAE .Additional visualization graphs such as histogram, boxplot, scatter plot, heatmapfeature importance graph, and actual vs predicted plot were used for better understanding and analysis.

As required, classification metrics such as Confusion Matrix, Precision Score, Recall Score, and F1 Score were also included by converting salary into salary categories and applying Logistic Regression for extended evaluation.

Overall, this project helps in understanding how different employee-related factors influence salary and provides a practical machine learning solution for fair, efficient, and accurate salary decision-making in organizations.

- **Steps of Implementation:**

The project follows a structured machine learning pipeline consisting of the following phases:

1.**Data Collection / Generation-**

Instead of relying on the synthetic data, the dataset is collected from the Kaggle platform (already mentions Kaggle as a source). Kaggle provides real-world datasets related to employee salary and job attributes.

The dataset used in this project contains 20 features such as Experience (years) ,Education Level,Job Role ,Department ,Skills ,Performance Rating ,Company Size,Location ,Salary (target variable) etc.The dataset is downloaded in CSV format from Kaggle and loaded into Python using Pandas.
In case of missing attributes or to simulate additional conditions, synthetic data generation is also performed to enhance the dataset by manually in the excel and then csv is imported .

2. Exploratory Data Analysis (EDA)

After loading data in step-1,now we understanding, cleaning, and analysis of the dataset to extract meaningful insights and prepare it for building the Employee Salary Prediction model.

• Data Understanding-

- The dataset was explored using:
  - head() → to view sample employee records
  - info() → to check data types and missing values
  - describe() → to analyze salary statistics (average salary, min, max, etc.)
- This helped us to understand how different features like experience and education are distributed and how they relate to salary.

• Data Cleaning-

- The dataset was cleaned to ensure accuracy:
  - Missing values were identified using isnull() and handled using dropna() or fillna() .
  - Duplicate employee records were removed using drop_duplicates() .
  - Inconsistent entries in features like job role and education were corrected .
- This step ensured that salary prediction is not affected by incomplete or incorrect employee data.

• **Outlier Detection and Handling-**

- Outliers (extremely high or low salary values) were detected using statistical methods like IQR:
  - Q1 = data.quantile(0.25)
  - Q3 = data.quantile(0.75)
  - IQR = Q3 - Q1
- Values outside (Q1 - 1.5*IQR) and (Q3 + 1.5*IQR) were treated as outliers.
- Unrealistic salary values or abnormal experience entries were removed or adjusted to prevent the model from learning incorrect patterns.

• **Data Analysis & Visualization-**

- Visualization techniques were used to understand relationships:

```
plt.hist(df["salary"])
plt.title("Salary Distribution")
plt.show()
plt.scatter(df["experience"], df["salary"])
plt.show()
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()
```

  - Histogram → to observe salary distribution
  - Scatter plot → to analyze how experience affects salary
  - Heatmap → to identify important features influencing salary
- We observed that experience and performance had a strong impact on salary, which helped in feature selection.

• **Outcome of EDA-**

- Identified key factors affecting employee salary (e.g., experience, education)
- Removed missing values, duplicates, and outliers from employee dataset .
- Understood relationships between features and salary .
- Prepared a clean and structured dataset for building the salary prediction model

3. Data Preprocessing-

In this step, the cleaned dataset was transformed into a format suitable for machine learning models. This includes converting categorical data into numerical form, scaling numerical features, and preparing data for training.

• **Handling Categorical Data-**

- Machine learning models require numerical input, so categorical features were converted into numeric values.
- This was done using encoding methods such as:
  - data['JobRole'].astype('category').cat.codes
- Other categorical columns like education and department were also encoded in a similar way.

• **Feature Scaling-**

StandardScaler

Standardization was applied to numerical features.

```
from sklearn.preprocessing import StandardScaler
scaler_std = StandardScaler()
X_std = scaler_std.fit_transform(X)
```

This improves model stability and reduces bias caused by large feature values.

MinMaxScaler

Normalization was also applied using MinMaxScaler.

```
from sklearn.preprocessing import MinMaxScaler
scaler_mm = MinMaxScaler()
X_mm = scaler_mm.fit_transform(X)
```

This ensures all features are in the same range and improves model performance.

• **Splitting Dataset-**

```
X_train, X_test, y_train, y_test = train_test_split(X_mm, y, test_size=0.2)
```

- The dataset was divided into training and testing sets using train_test_split().
- This helps in evaluating model performance on unseen data.

• **Verifying Preprocessed Data-**

- After preprocessing, the dataset was checked using data.head() and data.shape.
- This ensures that all transformations were applied correctly and the dataset is ready for model training.

• **Outcome of Data Preprocessing-**

- All categorical features converted into numerical form

- Numerical features scaled properly
- Dataset divided into training and testing sets
- Data prepared for regression model implementation.

4. Feature Selection-

In this step, only the most relevant features were selected from the dataset to improve model performance and reduce unnecessary complexity.

• Removing Redundant Features-

- Features that provide similar information or are not useful for prediction were removed.
- This helps in reducing duplication and simplifying the dataset.

• Applying Forward Selection-

- Forward selection is a step-by-step method where features are added one at a time based on their importance.
- The process starts with no features, and then features are added if they improve the model performance.
- Steps followed:
    1. Start with an empty feature set
    2. Train the model using one feature at a time
    3. Select the feature that gives the best performance (based on score)
    4. Add the selected feature to the model
    5. Repeat the process by adding remaining features one by one
    6. Stop when no further improvement is observed
- This can be implemented using sklearn:
    - from sklearn.feature_selection import SequentialFeatureSelector
    - Apply it with a regression model like LinearRegression()

Code-

```
model = LinearRegression()
sfs = SequentialFeatureSelector(model, n_features_to_select=10, direction="forward")
```

• Selecting Final Feature Set-

- After applying forward selection, only the most important features were retained.
- The final selected features were verified using data.columns.

5. Model Development-

In this step, regression models were implemented to learn the relationship between selected features and the target variable (salary).

• **Applying Linear Regression-**

- Linear Regression was used as the base model to understand the relationship between input features and salary.
- It was implemented using LinearRegression() from sklearn.
- Code-

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(X_train_sfs, y_train)
```

• **Applying Ridge Regression-**

- Ridge Regression was applied to reduce overfitting using regularization.
- It controls large coefficient values and improves model generalization.
- Code-

```
from sklearn.linear_model import Ridge
ridge = Ridge()
ridge.fit(X_train_sfs, y_train)
```

• **Applying Lasso Regression-**

- Lasso Regression was used to perform regularization and automatic feature selection.
- It can shrink some coefficients to zero, removing less important features.
- Code-

```
from sklearn.linear_model import Lasso
lasso = Lasso()
lasso.fit(X_train_sfs, y_train)
```

6. Model Evaluation-

All models were evaluated using multiple metrics.

Code -

```
from sklearn.metrics import r2_score
```

```
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
r2 = r2_score(y_test, pred)
rmse = np.sqrt(mean_squared_error(y_test, pred))
mae = mean_absolute_error(y_test, pred)
```

R^2 Score-

Measures model accuracy

Higher value = better model

RMSE-

Measures prediction error

Lower value = better

MAE-

Average absolute difference between actual and predicted salary

7. Confusion Matrix and Classification Metrics-

Code-

```
cm = confusion_matrix(y_test_c, pred_class)
precision = precision_score(y_test_c, pred_class, average="weighted")
recall = recall_score(y_test_c, pred_class, average="weighted")
f1 = f1_score(y_test_c, pred_class, average="weighted")
```

Precision

Correct positive predictions

Recall

Correctly identified actual positives

F1 Score

Balance between precision and recall

Confusion Matrix

Shows correct and incorrect classifications

This provides additional evaluation as required by the project.

8. Best Model Selection-

Among:

- Linear Regression
- Ridge Regression
- Lasso Regression

Ridge Regression performed best based on:

- Highest R^2 Score
- Lowest RMSE

Therefore, Ridge Regression was selected as the final model.

9. Final Prediction-

The final salary prediction was performed using Ridge Regression because it gave the highest R^2 Score and lowest RMSE.

Code -

```
prediction = ridge.predict(X_test_sfs[:1])

print(prediction)
```

The trained Ridge Regression model predicts salary for new employee input data.

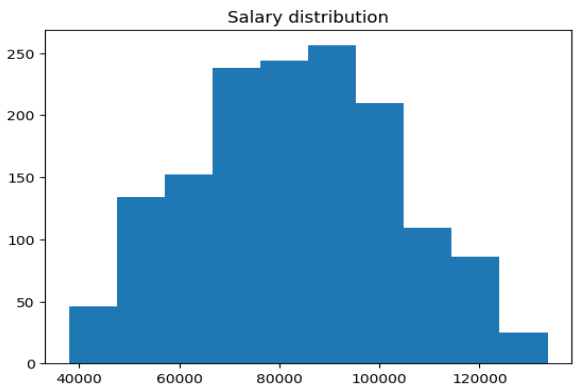
This demonstrates the practical implementation of machine learning for salary prediction systems.

- **Output analysis-**

This section contains all graphs and output visualizations used in the project.

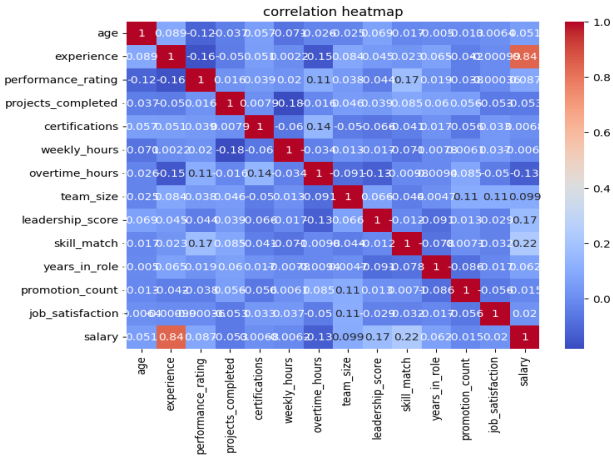
Graphs Used:

1. Histogram Before Scaling -

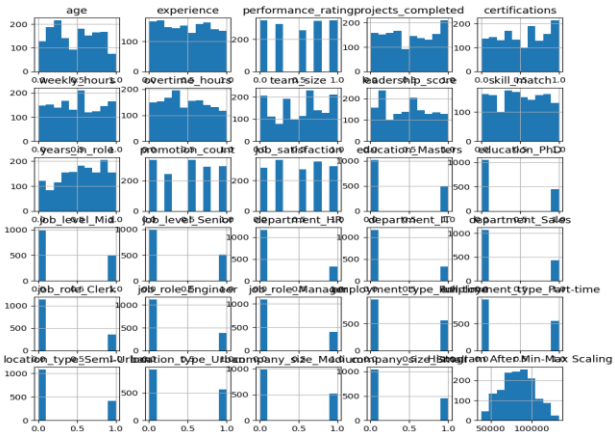


2.Scatter Plot-

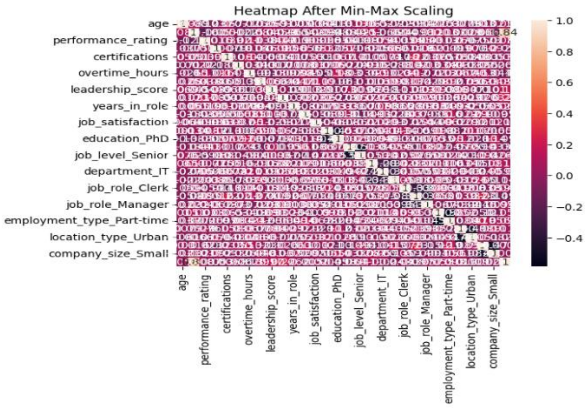
3.Correlation Heatmap –



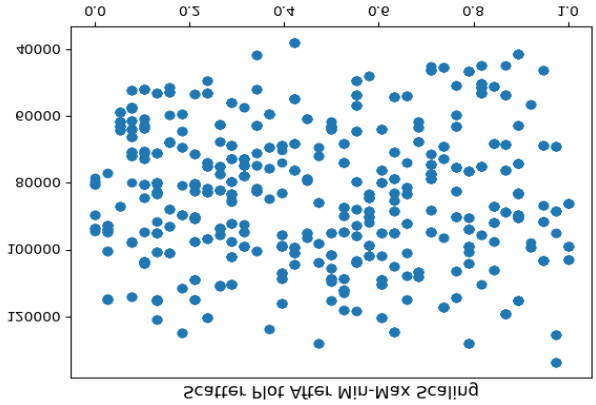
4.Histogram After Scaling –



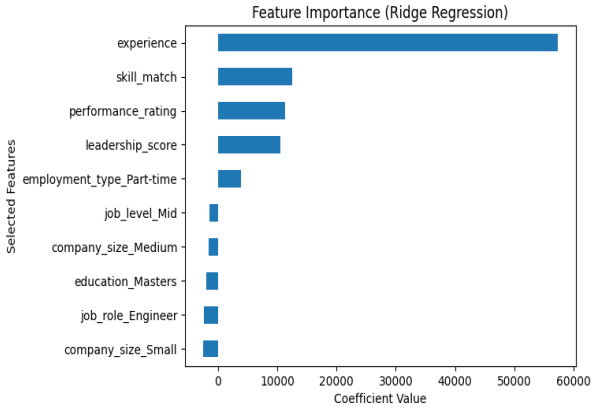
5.Heatmap After Scaling –



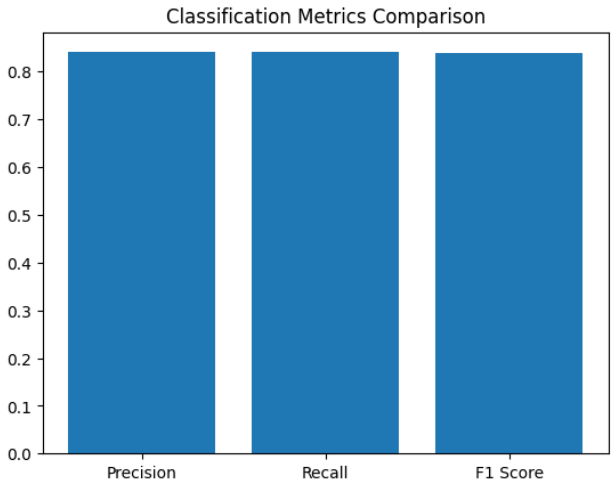
6.Scatter After Scaling-



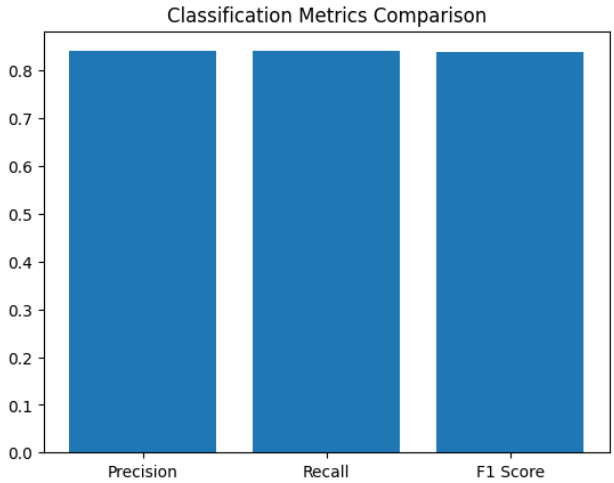
7.Feature Importance Graph –



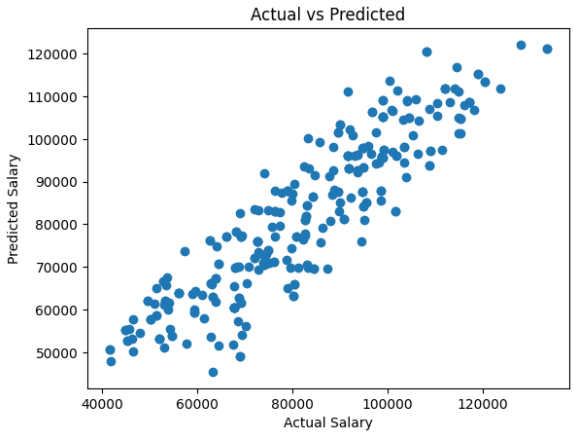
8.Classification Matrix Graph-



9.Model Comparison Graph-



10.Actual vs Predicted plot-



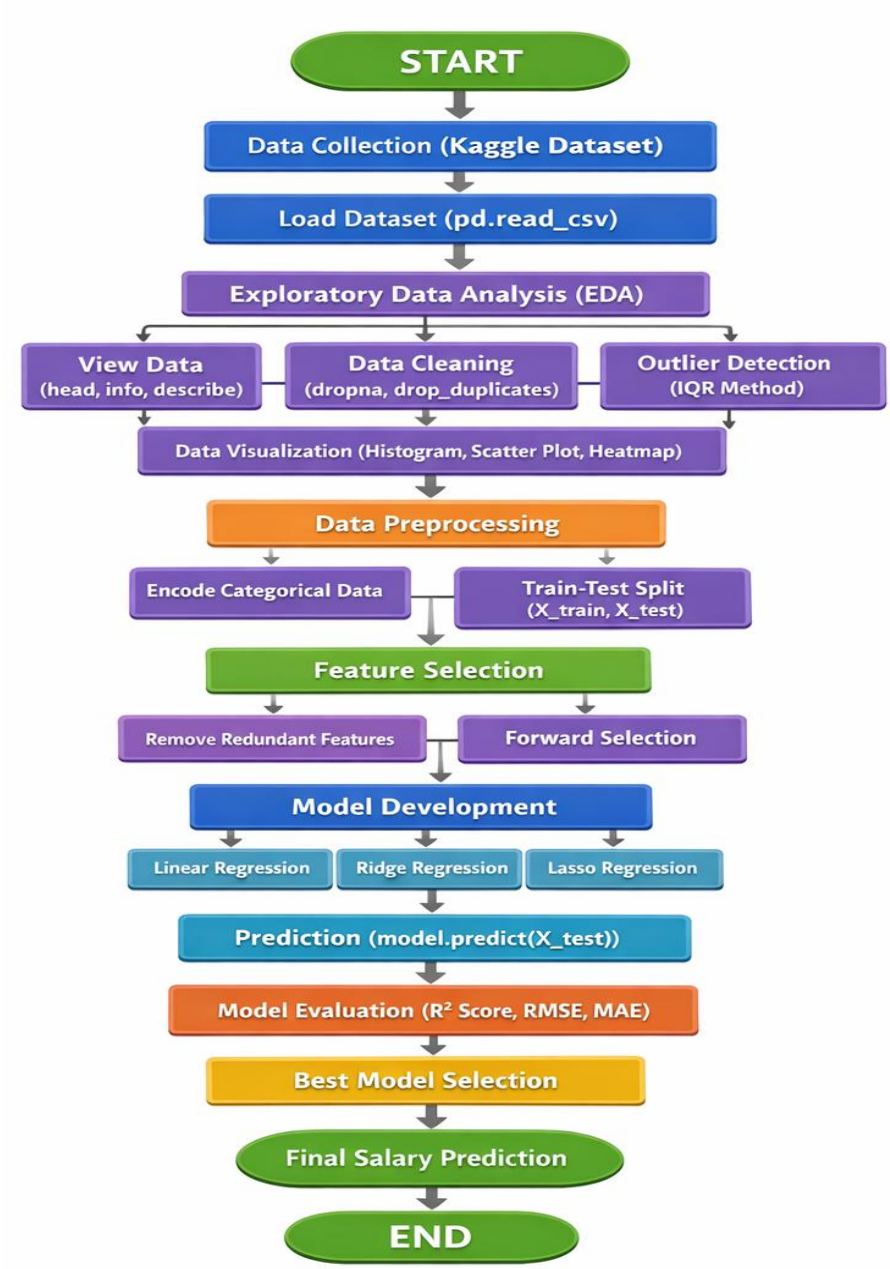


Fig. 1 Architecture Diagram

- **Expected Modules/ Libraries to be used :**

1. **NumPy-**
   NumPy is a fundamental library used for numerical computations in Python. It provides efficient array structures and mathematical functions required for data generation and manipulation. It is widely used for handling large datasets and performing complex calculations.

2. **Pandas-**
   Pandas is used for data manipulation and analysis through its DataFrame structure. It helps in organizing, filtering, and processing structured data efficiently. It also provides functions for reading and writing datasets in formats like CSV.

3. **Matplotlib-**
   Matplotlib is a visualization library used for creating basic plots such as histograms, line graphs, and scatter plots. It helps in understanding data trends and patterns visually. It is essential for representing analysis results graphically.

4. **Seaborn-**
   Seaborn is built on top of Matplotlib and is used for advanced statistical visualizations. It provides better-designed plots such as heatmaps and pair plots. It helps in identifying relationships and correlations between variables.

5. **Scikit-learn-**
   Scikit-learn is a machine learning library used for implementing algorithms and models. It provides tools for preprocessing, feature selection, model training, and evaluation. It simplifies the development of machine learning pipelines.

- **Expected Evaluation metrics to be used:**

To evaluate the performance of the regression models, multiple metrics and visualization techniques are used to ensure accurate and reliable results.

1. **R^2 Score-**

R^2 score measures how well the regression model explains the variability of the dependent variable (salary). It indicates the proportion of variance captured by the model. A higher value represents better model performance.

2. **RMSE (Root Mean Squared Error)-**

RMSE measures the average magnitude of prediction errors by taking the square root of squared differences. It gives more importance to larger errors, making it useful for detecting significant deviations. Lower RMSE indicates better accuracy.

3. **MAE (Mean Absolute Error)-**

MAE calculates the average absolute difference between predicted and actual values. It provides a simple and interpretable measure of error. It is less sensitive to outliers compared to RMSE.

4. **Precision-**

Precision measures how many of the predicted positive values are actually correct. It is used in classification problems and helps evaluate the quality of positive predictions.

5. **Recall-**

Recall measures how many actual positive values were correctly identified by the model. It helps evaluate the completeness of classification predictions.

6. **F1 Score-**

F1 Score is the harmonic mean of Precision and Recall. It provides a balanced evaluation when both precision and recall are important.

7. **Confusion Matrix-**

A confusion matrix is used to evaluate classification performance. It shows correct and incorrect predictions in tabular form and helps understand how well the model classified salary categories.

- **Required topics from the subject:**

1. **Data Preprocessing-**
   Data preprocessing is used to prepare the dataset for machine learning. It involves organizing, formatting, and transforming raw data into a structured form. This step ensures that the dataset is clean and suitable for further analysis.

2. Data Cleaning-
Data cleaning is performed to handle missing or inconsistent values in the dataset. Techniques such as filling missing values using mean/mode and removing unwanted data are applied. This improves data quality and model performance.

3. Outlier Detection (IQR Method)-
Outliers are extreme values that can negatively affect model accuracy. The Interquartile Range (IQR) method is used to detect and remove such values. This helps in making the model more robust and reliable.

4. Feature Scaling (StandardScaler)-
Feature scaling is used to normalize numerical data so that all features have similar ranges. This prevents bias in model learning due to large value differences. It improves convergence and accuracy of regression models.

5. Train-Test Split-
The dataset is divided into training and testing sets to evaluate model performance. The training set is used to train the model, while the testing set is used to check its accuracy. This ensures proper validation of the model.

6. Linear Regression-
Linear Regression is used to establish a relationship between input features and salary. It predicts salary as a continuous value based on independent variables. It is one of the simplest and widely used regression techniques.

7. Ridge Regression
Ridge Regression is an improved version of Linear Regression that uses regularization to reduce overfitting. It controls large coefficient values and improves model generalization, making predictions more reliable.

8. Lasso Regression
Lasso Regression also uses regularization and performs automatic feature selection by shrinking some coefficients to zero. This helps remove less important features and improves model simplicity.

9. Logistic Regression
Logistic Regression is used for classification problems. In this project, it was used after converting salary into categories such as Low, Medium, and High. It helped calculate classification metrics like confusion matrix and F1-score.

10.Evaluation metrices-

Evaluation metrics are used to measure the performance and accuracy of machine learning models.R^2 Score, RMSE, MAE, and MSE are used for regression model evaluation, while Precision, Recall, F1 Score, and Confusion Matrix are used for classification evaluation.

11. Prediction-

Prediction is the process of using the trained model to estimate salary for new input data. The model takes employee features as input and generates a salary output. This demonstrates the practical use of the machine learning model.

- **Platform Required:**

1. **Jupyter Notebook-**

Jupyter Notebook is used for writing, executing, and testing code in an interactive environment. It allows combining code, outputs, and visualizations in one place. This makes it ideal for data analysis and experimentation.

2. **Python (IDLE / VS Code)-**

Python is the main programming language used for implementing the project. IDEs like IDLE and VS Code provide an environment for writing, debugging, and running code efficiently. They support various libraries required for machine learning.

3. **GitHub-**

GitHub is used for version control and managing the project repository. It allows multiple team members to collaborate and track changes in the code. It also helps in organizing the project into phases and sharing it with others.

- **Books and Link Sources:**

1. **Python for Data Analysis –** Wes McKinney

2. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow –** Aurélien Géron

3. **https://scikit-learn.org -** for official documentation of machine learning models and preprocessing.

4. **https://www.kaggle.com -** for dataset and practice.

5. **https://pandas.pydata.org -** for official documentation of data analysis and manipulation.